KI/SZZVP Aplikovaná informatika: volitelné bloky
Programování a softwarové inženýrství

I. Objektově orientované návrhové vzory 1
Ukázková úloha I.1. Návrhové vzory ve správci kontaktů 2
Ukázková úloha I.2. Návrhové vzory v aplikaci typu studijní systém 2
II. Vývoj desktopové aplikace 3
Ukázková úloha II.1. Generování fraktálního stromu 4
Ukázková úloha II.2. Aplikace pro ukládání výsledků do databáze 4
III. Vývoj mobilní aplikace 5
Ukázková úloha III.1. Čtečka RSS souborů 5
IV. Vývoj webové aplikace 5
Ukázková úloha IV.1. Vývoj webového zápisníčku úkolů s využitím XML 6
Ukázková úloha IV.2. Vývoj skladového systému s využitím relační databáze 6
V. Objektově orientovaný návrh 7
Ukázková úloha V.1. Návrh rezervačního systému pro malé hotely a penziony 8
Ukázková úloha V.2. Systém správy projektů pro malé týmy 8

I. Objektově orientované návrhové vzory

Předměty: KI/OON, KI/OONV, KI/SWI

Popis tématu:
Vytvořte objektově-orientovanou konzolovou aplikaci v jazyce C#, ve které implementuje 3 návrhové vzory pro řešení úkolů.

Vyžadované znalosti:
Typový systém ve staticky typovaných objektových programovacích jazycích, polymorfismus založený na sdíleném rozhraní, polymorfismus založený na dědičnosti (specifikátory přístupu), návrhové vzory (principy a cíle), vytvářející návrhové vzory (Tovární metody a objekty, Jedináček), strukturální návrhové vzory (Adaptér, Dekorátor, Most, Muší váha), vzory týkající se chování (Příkaz, Pozorovatel, Memo), diagramy tříd, návrh SW architektury, nástroje pro kontrolu verzí, testování (funkční), validace a verifikace

Doporučené materiály a aplikace:
1. Jazyk C#, Diagrams.net (dříve draw.io), plantUML
2. Tutoriály ze stránek:
   - https://refactoring.guru/design-patterns
   - https://www.w3schools.com/cs/index.php

Ukázková úloha I.1. Návrhové vzory ve správci kontaktů

Popis úlohy:
Vytvořte aplikaci pro správu kontaktů, která umožní uživatelům ukládat, prohlížet, upravovat a mazat kontaktní informace. Při vývoji aplikace je třeba použít následující návrhové vzory:

1. Prototype: Pro efektivní vytváření a kopírování kontaktních záznamů.
2. Command: Pro zaznamenávání a možné vrácení změn v kontaktech.
3. Iterator: Pro navigaci skrze kolekci kontaktů.

Požadavky na odevzdání:
Řešitel odevzdá:

1. Funkční konzolová aplikace splňující níže uvedené funkční požadavky.
2. Zdrojový kód aplikace v repozitáři Github nebo Gitlab.
3. Stručná uživatelská příručka s návodem na použití aplikace.

Uživatelské požadavky:

1. Funkční požadavky:
    - Implementace přidávání nových kontaktů s detaily jako jméno, telefonní číslo, e-mail, atd.
    - Implementace prohlížení, vyhledávání, úpravy a smazání existujících kontaktů.
    - Možnost ukládání kontaktů do externího zdroje (stačí jednoduchý soubor).
    - Načítání kontaktů při spuštění aplikace.
2. Implementace návrhových vzorů:
    - Využití Prototype pro efektivní vytváření a kopírování kontaktů.
    - Implementace Command pro zaznamenávání a možnost vrácení změn v kontaktech.
    - Použití Iterator pro navigaci skrze seznam kontaktů.
3. Uživatelské rozhraní:
    - Přehledné a intuitivní konzolové rozhraní pro jednoduchou navigaci a ovládání aplikace.
4. Dokumentace a komentáře:
    - Stručná dokumentace kódu s popisem použitých návrhových vzorů.
    - Komentáře v kódu pro lepší pochopení struktury a logiky programu.
5. Testování:
    - Implementace jednotkových testů pro klíčové funkce aplikace.

Ukázková úloha I.2. Návrhové vzory v aplikaci typu studijní systém

Popis úlohy:
Vytvořte konzolovou aplikaci v jazyce C#, která simuluje základní funkce studijního systému. Aplikace by měla umožnit registraci studentů, správu kurzů, přihlašování na kurzy a zobrazení studijních výsledků. Během vývoje aplikace je třeba použít následující tři návrhové vzory:

1. Factory Method: Implementujte pro vytváření různých typů uživatelských účtů (student, učitel, správce) nebo kurzů.
2. Observer: Použijte pro informování studentů o změnách v kurzech nebo rozvrzích.
3. Strategy: Použijte pro změnu algoritmu výpočtu pro různé metody hodnocení studentů (aritmetický průměr, vážený průměr apod.).

Požadavky na odevzdání:
Řešitel odevzdá:

1. Funkční konzolová aplikace splňující níže uvedené funkční požadavky.
2. Zdrojový kód aplikace v repozitáři Github nebo Gitlab.
3. Stručná uživatelská příručka s návodem na použití aplikace (stačí ve formě README.md souboru).

Uživatelské požadavky:

1. Funkcionality:
    - Možnost registrace a přihlašování uživatelů (studentů, učitelů).
    - Správa kurzů včetně možnosti přidávání a upravování informací o kurzech.
    - Přihlašování studentů na kurzy a správa jejich studijního plánu.
    - Zobrazení studijních výsledků a průměrných známek.
2. Implementace Návrhových Vzorů:
    - Použití Observer vzoru pro sledování změn v kurzech a informování studentů.
    - Použití Factory Method vzoru pro vytváření různých typů účtů a kurzů.
    - Použití Strategy vzoru pro flexibilní způsoby hodnocení studentů.
3. Uživatelské Rozhraní:
    - Přehledné a intuitivní konzolové rozhraní pro jednoduchou navigaci a ovládání aplikace.
4. Dokumentace a komentáře:
    - Stručná dokumentace kódu s popisem použitých návrhových vzorů.
    - Komentáře v kódu pro lepší pochopení struktury a logiky programu.
5. Testování:
    - Implementace jednotkových testů pro klíčové funkce aplikace.

II. Vývoj desktopové aplikace

Předměty: KI/GUI, KI/OON, KI/SWI

Popis úlohy:
Vytvořte aplikaci obsahující grafické uživatelské rozhraní v C#, nebo Pythonu. Aplikace používá událostmi řízené programování. Předpokládá se využití objektově orientovaného programování. Aplikace pracuje s persistentními informacemi v databázi, případně XML/JSON souboru a využívá asynchronicity. Součástí aplikace je i jednoduchá dokumentace vytvořená pomocí dokumentačních komentářů a UML diagramů.

Vyžadované znalosti:
Použití základních komponent v ekosystému C# (WPF+XAML), či Python (PyQT, TkInter). Použití událostmi řízeného programování. Schopnost vytvořit jednoduchou databázi a připojit k ní aplikaci (ORM, nebo SQL dotazy). Znalost ukládání a čtení XML/JSON v daném ekosystému. Znalost použití asynchronicity, resp. tvorby vícevláknových aplikací.

Doporučené materiály a aplikace:
Vývojové prostředí – VisualStudio, Pycharm, VSCode apod. (dle konkrétního zadání)
Dokumentace WPF, PyQT, Tkinter, ORM frameworky

Ukázková úloha II.1. Generování fraktálního stromu

Popis úlohy:
Vytvořte aplikaci s grafickým uživatelským rozhraním v jazyce C#, která umožňuje generování fraktálního stromu (https://en.wikipedia.org/wiki/Fractal_canopy) a jeho vizualizaci.

Řešitel odevzdá:
1. Funkční aplikaci splňující níže uvedené funkční požadavky.
2. Dokumentovaný (!) zdrojový kód aplikace v repozitáři Github nebo Gitlab.
3. Stručná uživatelská příručka s návodem na použití aplikace.

Funkční požadavky:
1. Aplikace umožní zadat základní parametry stromu (počet iterací, úhel, koeficient zkracování, výběr barvy pro kreslení).
2. Aplikace umožní výpočet zahájit, přerušit, pozastavit a znovu rozběhnout. Aplikace reaguje na události i v průběhu výpočtu stromu.
3. Aplikace umožní exportovat vytvořený strom exportovat do JSON souboru a umožní jej načíst.

Ukázková úloha II.2. Aplikace pro ukládání výsledků do databáze

Popis úlohy:
Vytvořte aplikaci s grafickým uživatelským rozhraním v jazyce Python pro správu výsledků modelů strojového učení. Aplikace umožní založit experiment (řešená úloha) a model, jeho konfiguraci a výsledky.
Řešitel odevzdá:
1. Funkční aplikaci splňující níže uvedené funkční požadavky.
2. Dokumentovaný (!) zdrojový kód aplikace v repozitáři Github nebo Gitlab.
3. Stručná uživatelská příručka s návodem na použití aplikace.
4. Stručná dokumentace k databázi

Funkční požadavky:

1. Aplikace umožní zadání jména experimentu a jeho jednoduchého textového popisu.
2. Aplikace umožní zadání typu modelu (RandomForrest, SVM/SVC), jeho jména, popisu a některých jeho parametrů (nejméně tří). Předpokládejme rozhraní scikitlearn (https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html, https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html)
3. Aplikace umožní přiřadit vytvořený model do experiementu a přiřadit mu výsledky (předpokládejme, že je to jedno číslo).. V jednom experimentu může být více modelů.
4. Aplikace umožní v rámci experimentu filtrovat modely dle typu a setřídit dle výsledků.
5. Aplikace ukládá experimenty a modely do databáze.

Datová vrstva: Využijte relační databázi (např. MySQL) pro ukládání a správu dat.

III. Vývoj mobilní aplikace

Předměty: KI/PMP, KI/OON, KI/SWI

Popis úlohy:
Vytvoření jednoduché aplikace pro Android, s dvěma aktivitami a činností na pozadí (přístup k webovým službám, využití senzorů včetně pozičních služeb, ukládání dat do databáze)

Výstupy:
Git projekt se všemi zdrojovými soubory a navíc:

- UML diagramy pro vlastní logiku programu
- dokumentace pro vývojáře (může být generována z anotovaných zdrojových kódů)
- dokumentace pro uživatele (popis chování programu)

Vyžadované znalosti:
Java (základy) resp. C#, Android SDK, UML, zpracování JSON a XML, relační databáze (SQLite)

Ukázková úloha III.1. Čtečka RSS souborů

Požadavky na odevzdání:
Řešitel odevzdá:

1. Funkční mobilní aplikaci splňující níže uvedené funkční požadavky.
2. Zdrojový kód aplikace v repozitáři Github nebo Gitlab (s dokumentací)
3. Stručná uživatelská příručka s návodem na použití aplikace.

Funkční požadavky:

menu či kontextové tlačítko pro načtení RSS kanálu z veřejného zdroje (např. https://www.idnes.cz/rss)

seznamové zobrazení zpráv (nejlépe RecyclerView)

po zvolení konkrétní zprávy zobrazení detailní informace

persistentní ukládání zpráv (zprávy se zobrazují i v případě offline režimu, ale jen ty časově relevantní)

automatická aktualizace v určitém časovém intervalu

IV. Vývoj webové aplikace

Předměty: KI/PRI, KI/SWI

Popis tématu:
Vytvořte webovou aplikaci s databází, která řeší podnikový problém.

Vyžadované znalosti:
Značkovací jazyk XML, Specifikace XML, Zobrazení XML souborů, HTML DOM a XML DOM, Javascript a HTML, Javascript a XML DOM, Jazyk PHP, PHP (ukládání dat, Cookie a session), PHP (grafika), PHP (práce s objekty), PHP a XML DOM, PHP (simpleXML), PHP

(práce s databázemi), Analýza požadavků, Návrh SW architektury, Nástroje pro kontrolu verzí, Testování, validace a verifikace, problematika API, Nasazení do provozu a podpora.

Doporučené materiály a aplikace:

3. HTML5, CSS3, Bootstrap5, Tailwind, PHP8, JS (ES14), TS5 + případně libovolný framework pro PHP nebo JS. Povinný je jen jazyk PHP.
4. Tutoriály ze stránek:
   - https://www.w3schools.com/php/default.asp
   - https://www.w3schools.com/xml/

Ukázková úloha IV.1. Vývoj webového zápisníčku úkolů s využitím XML

Popis úlohy:
Vytvořte webovou aplikaci, která slouží jako zápisník úkolů v jazyce PHP. Data budou ukládána do XML souborů.

Požadavky na odevzdání:
Řešitel odevzdá:

1. Funkční webovou aplikaci splňující níže uvedené funkční požadavky.
2. Zdrojový kód aplikace v repozitáři Github nebo Gitlab.
3. Stručná uživatelská příručka s návodem na použití aplikace.

Uživatelské požadavky:

1. Funkcionality:
   - Implementujte uživatelské přihlášení a registraci pro personalizaci úkolů (využijte XML jako databázi). Ujistěte se, že osobní údaje uživatelů jsou bezpečně zpracovávány a uloženy.
   - Aplikace by měla umožňovat snadné přidávání, editaci a mazání úkolů. Rozhraní by mělo být intuitivní a vkusné (využijte stylová pravidla).
   - Implementujte funkce pro kategorizaci úkolů (např. práce, osobní, studium).
   - Implementujte možnost filtrování úkolů podle kategorií nebo stavu (např. nezahájené, zahájené, dokončené).
   - Implementujte možnost importování existujícího úkolu ve formátu XML do zápisníku přes formulář.
2. Data a XML:
   - Všechny úkoly a jejich detaily uložte ve formátu XML.
   - Implementujte funkce pro čtení a zápis do XML souborů.
   - Zajistěte, aby data byla dobře strukturovaná a snadno přístupná pro další zpracování.
   - Vytvořte XSD soubor, který slouží pro validaci XML souborů, reprezentujících úkol. Tento soubor bude použit pro validaci importovaného XML souboru s úkolem do zápisníku.

Ukázková úloha IV.2. Vývoj skladového systému s využitím relační databáze

Popis úlohy:
Vytvořte webovou aplikaci pro správu skladu v jazyce PHP. Data budou ukládána do relační databáze.

Požadavky na odevzdání:
Řešitel odevzdá:

4. Funkční webovou aplikaci splňující níže uvedené funkční požadavky.
5. Zdrojový kód aplikace v repozitáři Github nebo Gitlab.
6. Stručná uživatelská příručka s návodem na použití aplikace.

Uživatelské požadavky:

1. Funkčnost:
    - Implementujte funkce pro přidávání, upravování, mazání a prohlížení produktů ve skladu. Kontext skladového systému je na Vás (např.: stavebnictví, strojírenství, gastronomie).
    - Vytvořte funkce pro správu objednávek, včetně možnosti přidávat nové objednávky a sledovat jejich stav.
    - Implementujte systém pro sledování stavu zásob a upozornění na nízké stav zásoby.
    - Implementujte API pro integraci s externími systémy nebo aplikacemi.
2. Relační Databáze:
    - Využijte relační databázi (např. MySQL) pro ukládání a správu dat.
    - Navrhněte schéma databáze vhodné pro skladovací systém, včetně tabulek pro produkty, objednávky, zásoby a uživatele.
3. Bezpečnost:
    - Implementujte zabezpečení proti běžným webovým hrozbám (např.: SQL injection).
    - Zabezpečte uživatelské data a přístup k systému pomocí vhodných rolí. Vytvoření uživatelských rolí a oprávnění pro různé typy uživatelů (např. správce, skladník).

V. Objektově orientovaný návrh

Předměty: KI/OON, KI/SWI

Popis tématu:
Vytvořte systémovou dokumentaci aplikace pomocí jazyka UML a dodejte uživatelský návrh aplikace pomocí vhodně zvoleného softwaru pro drátěné modely nebo grafické prototypy.

Vyžadované znalosti:
Fáze tvorby software, Stavební bloky jazyka UML, diagram užití, diagram tříd, diagram objektů, diagram aktivit, stavový diagram, sekvenční diagram, komunikační diagram, tvorba wireframe modelů GUI, tvorba dokumentace, generování dokumentace, analýza požadavků, návrh SW architektury, problematika API

Doporučené materiály a aplikace:

5. PlantUML jazyk nebo aplikace Diagram.net (bývalé draw.io)
6. Tutoriály ze stránky: https://www.visual-paradigm.com/
7. Specifikace UML ze stránek OMG, např. https://www.omg.org/spec/UML/2.5.1/About-UML

Ukázková úloha V.1. Návrh rezervačního systému pro malé hotely a penziony

Popis úlohy:
Navrhněte aplikaci pro rezervaci pokojů v malých hotelech a penzionech. Aplikace bude zahrnovat funkce pro vyhledávání pokojů, rezervaci, správu profilu uživatele a možnost poskytování zpětné vazby na ubytování.

Požadavky na odevzdání:
Řešitel odevzdá softwarovou dokumentaci, která bude obsahovat systémový návrh ve formátu UML diagramů a snímky grafického prototypu ze softwaru Figma nebo drátěný model z libovolné grafické aplikace.

- Povinné UML diagramy: Diagram případů užití, Sekvenční diagram, Diagram Tříd.
- Návrh uživatelského rozhraní musí obsahovat alespoň: hlavní stránku, stránku rezervace, profil uživatele a formulář pro zpětnou vazbu. Návrh grafického rozhraní můžete realizovat ve formě webové, stolní nebo mobilní aplikace.

Uživatelské požadavky:

1. Vyhledávání a rezervace pokojů:
    - Umožnit uživatelům vyhledávat pokoje podle data, počtu osob, ceny a dalších specifikací (např. wi-fi, snídaně).
    - Poskytnout detailní informace o pokoji, včetně fotografií, popisu vybavení a recenzí od předešlých hostů.
    - Umožnit uživatelům provést rezervaci pokoje, vyplnit potřebné údaje (jako jméno, kontakt, případné požadavky) a potvrdit rezervaci.
2. Správa profilu uživatele:
    - Umožnit uživatelům vytvořit a spravovat svůj uživatelský profil, včetně osobních informací.
    - Nabídnout možnost změny hesla a osobních údajů.
    - Poskytnout přehled předchozích a naplánovaných rezervací s možností zrušení nebo úpravy rezervace.
3. Zpětná vazba a hodnocení:
    - Umožnit uživatelům hodnotit a psát recenze na ubytování.
    - Zobrazit hodnocení a recenze hotelu/penzionu.
    - Poskytnout hotelu/penzionu nástroj pro reagování na recenze a zpětnou vazbu.
4. Integrace platebního systému:
    - Nabídnout různé platební možnosti, jako jsou kreditní karty, bankovní převody a online platební platformy.
5. Uživatelské rozhraní a navigace:
    - Vytvořit intuitivní a přívětivé uživatelské rozhraní pro snadnou navigaci a používání aplikace.

Ukázková úloha V.2. Systém správy projektů pro malé týmy

Popis úlohy:
Navrhněte aplikaci pro správu projektů, která umožní uživatelům vytvářet projekty, přidávat úkoly, sledovat pokrok a spolupracovat s týmem.

Požadavky na odevzdání:
Řešitel odevzdá softwarovou dokumentaci, která bude obsahovat systémový návrh ve formátu UML diagramů a snímky grafického prototypu ze softwaru Figma nebo drátěný model z libovolné grafické aplikace.

- Povinné UML diagramy: Diagram Případů užití, Diagram aktivit a Stavový diagram.
- Návrh uživatelského rozhraní musí obsahovat alespoň: Návrh domovské stránky, stránky projektu, rozhraní pro přidání úkolů a nástěnky pro sledování pokroku.

Uživatelské požadavky:

1. Vytváření a správa projektů:
    - Umožnit uživatelům vytvářet nové projekty, nastavovat jejich názvy, popisy, termíny a přiřazovat týmové členy.
    - Poskytnout funkci pro editaci a aktualizaci detailů projektu.
    - Nabídnout možnost archivovat nebo smazat dokončené či neaktivní projekty.
2. Přidávání a správa úkolů:
    - Umožnit přidávat úkoly do projektů, včetně názvů, popisů, priorit, termínů a přiřazení konkrétním týmovým členům.
    - Poskytnout možnost editace, aktualizace a mazání úkolů.
    - Implementovat systém pro sledování stavu úkolů (např. "k provedení", "ve vývoji", "hotovo").
3. Kolaborační nástroje a komunikace:
    - Nabídnout integrovaný chat pro komunikaci mezi členy týmu uvnitř aplikace.
    - Umožnit připojování souborů a odkazů k úkolům a projektům.
4. Sledování pokroku a reporty:
    - Implementovat dashboard pro zobrazení aktuálního stavu projektů a úkolů.
    - Poskytnout nástroje pro generování reportů o pokroku, včetně grafů a statistik.
    - Umožnit exportování reportů ve formátech jako PDF nebo CSV.
5. Uživatelské rozhraní a navigace:
    - Vytvořit čisté, intuitivní uživatelské rozhraní pro snadnou orientaci a efektivní práci.
6. Správa uživatelů a bezpečnost:
    - Umožnit vytváření a správu uživatelských účtů, včetně rolí a oprávnění v rámci týmu.